Python Programming

Interview Preparation — Question Bank

Day 1 → Long Weekend · 2-Hour Discussion Session

Session Plan	Content	Questions	Time
Part 1 — MCQs	Web, Python Basics, Operators, Loops, Data Structures, Functions	40 MCQs	60 min
Part 2 — One-Liners	Quick recall Q&A across all topics	30 Q&A	30 min
Part 3 — Discussion	Review wrong answers, clarify doubts	Open	30 min
Total		**70 Questions**	**120 min**

Topics Covered
Day 1-2: Web basics — API, DNS, Domain, IP, HTTP methods, Status codes, Request/Response body, Client/Server, GitHub, Virtual environment, pip
Day 3: Python basics — Running files, Syntax errors, Variables, Data types, input(), type conversion, String concatenation
Day 4: Operators — Arithmetic, Comparison, Assignment, Logical (and/or/not), if/elif/else, Short-circuit evaluation
Day 5: Loops — for, while, range(), iterating lists, break, continue, while..else, infinite loops, random module
Day 6: Data structures — List methods, Tuple, Set operations, Dictionary — all methods, iterating with .items(), Functions — def, parameters, return, re

Note: Correct answers are highlighted in green ✓. Each MCQ includes an explanation. For One-Liners, try to answer before reading the answer. Time yourself!

PART 1 — Multiple Choice Questions

40 questions · Aim: 1.5 minutes per question · Total: ~60 minutes

Section 1 — Web & Internet Fundamentals

Days 1–2 | Q1–Q8

Q1. What does DNS stand for, and what is its primary job?

A. Domain Name Server — assigns IP addresses to computers	**B.** Domain Name System — translates domain names into IP addresses ✓
C. Dynamic Network Service — manages WiFi connections	**D.** Data Node System — stores website files

Explanation: DNS = Domain Name System. Its job is to translate human-readable domain names like google.com into machine-readable IP addresses like 18.x.x.x. Without DNS you'd have to memorise IP addresses for every website.

Q2. Which of the following is a famous public DNS resolver used for troubleshooting internet issues?

A. 192.168.1.1	**B.** 127.0.0.1
C. 8.8.8.8 ✓	**D.** 255.255.255.0

Explanation: 8.8.8.8 is Google's public DNS resolver. 1.1.1.1 is Cloudflare's. 192.168.1.1 is a typical router address. 127.0.0.1 is localhost (your own machine).

Q3. What is the difference between a client and a server?

A. Client stores data; server displays it	**B.** Client sends requests; server processes them and sends responses ✓
C. Client is hardware; server is software	**D.** They are the same thing — just different names

Explanation: A client initiates requests (browser, mobile app, Python script). A server receives and processes those requests and sends back responses. This is the fundamental client-server model the entire web is built on.

Q4. Which HTTP status code means the request was successful?

A. 404	**B.** 500
C. 200 ✓	**D.** 301

Explanation: 200 = OK (success). 404 = Not Found. 500 = Internal Server Error. 301 = Moved Permanently (redirect). These are the most common status codes you'll see when making API calls.

Q5. What format is most commonly used for API request and response bodies?

A. XML	**B.** CSV
C. HTML	**D.** JSON ✓

Explanation : JSON (JavaScript Object Notation) is the standard format for API data exchange. It uses key-value pairs like {'name': 'John', 'age': 25} and is supported natively by Python's requests library via response.json().

Q6. What is the purpose of a virtual environment in Python development?

A. To run Python code faster	**B.** To isolate project dependencies so different projects don't conflict ✓
C. To connect to the internet securely	**D.** To store Python files in the cloud

Explanation : Virtual environments isolate packages per project. Project A may need Django 3, Project B needs Django 5 — without isolation they conflict. Always activate venv before installing packages with pip.

Q7. Which HTTP method is used to RETRIEVE data from a server without changing anything?

A. POST	**B.** DELETE
C. PUT	**D.** GET ✓

Explanation : GET = read/fetch data. POST = create/send data. PUT = update data. DELETE = remove data. When you type a URL in a browser, you're making a GET request.

Q8. What command is used to install a Python package like 'requests'?

A. python install requests	**B.** install requests
C. pip install requests ✓	**D.** get requests

Explanation : pip is Python's package manager. 'pip install requests' downloads and installs the requests library. Always activate your virtual environment first so packages install into the project, not globally.

Section 2 — Python Basics & Data Types

Day 3 | Q9–Q16

Q9. What is the output of the following code?

```
print(10 / 3)
```

A. 10	**B.** 10.0
C. 3.3333333333333335 ✓	**D.** Error

Explanation : In Python 3, the / operator always returns a float even for whole numbers. 10 / 3 = 3.333... Use // for floor division: 10 // 3 = 3.

Q10. What will type(x) return if x = '42'?

A. <class 'int'>	**B.** <class 'float'>
C. <class 'str'> ✓	**D.** <class 'bool'>

Explanation : Quotes make it a string. '42' is a string, not a number. int('42') would convert it. input() always returns a string — this is why you must use int(input()) when expecting numbers.

Q11. Which of these will cause a SyntaxError?

A. print('Hello')	**B.** print(Hello) ✓
C. x = 'Hello'	**D.** print('Hello', 'World')

Explanation : print(Hello) without quotes means Python looks for a variable named Hello. Since it doesn't exist as a variable AND it's not in quotes as a string, it causes a NameError (not SyntaxError technically — but is the 'wrong' usage). The syntax rule is: strings must be in quotes.

Q12. What is the output?

```
print('hello' + 5)
```

A. hello5	**B.** Error: can only concatenate str to str ✓
C. hello 5	**D.** 55

Explanation : You cannot use + to join a string and an integer. Python raises: TypeError: can only concatenate str (not 'int') to str. Fix: use str(5) or f'hello{5}' or print('hello', 5).

Q13. What does input() ALWAYS return, regardless of what the user types?

A. int	**B.** float
C. str ✓	**D.** The same type as what was typed

Explanation : input() always returns a string — even if the user types a number. This is why int(input()) is needed when expecting numeric input. Forgetting this causes TypeErrors when doing math.

Q14. Is Python case-sensitive? What is the output of this code?

```
a = 'hello'
print(A)
```

A. No — prints 'hello'	**B.** Yes — NameError: name 'A' is not defined ✓
C. Yes — prints 'hello' in uppercase	**D.** No — both A and a refer to same variable

Explanation : Python is case-sensitive. 'a' and 'A' are completely different variable names. Assigning a = 'hello' and then printing A causes a NameError because A was never defined.

Q15. What is the output of this type conversion?

```
print(float('3.5'))
```

A. 3.5 ✓	**B.** 3
C. Error: cannot convert float string	**D.** 4

Explanation : float('3.5') converts the string '3.5' to a float. int('3.5') would fail because '3.5' is not a valid integer string. Only int('3') would work for integer conversion.

Q16. In Python, there is NO separate 'char' data type. A single character like 'A' is treated as:

A. A character type	**B.** An integer (ASCII value)
C. A string of length 1 ✓	**D.** A boolean

Explanation : Unlike C or Java, Python has no char type. 'A' is simply a string of length 1. type('A') returns <class 'str'>. This is one of Python's simplifications compared to other languages.

Section 3 — Operators & Conditionals

Day 4 | Q17–Q24

Q17. What is the output of 17 % 5?

A. 3.4	**B.** 3
C. 2 ✓	**D.** 0

Explanation: Modulus (%) returns the REMAINDER after division. 17 ÷ 5 = 3 remainder 2. So 17 % 5 = 2. Quick rule: 5 × 3 = 15, 17 - 15 = 2.

Q18. What is the difference between = and ==?

A. No difference — they do the same thing	**B.** = compares, == assigns
C. = assigns a value to a variable, == compares two values ✓	**D.** = is used in functions, == is used in loops

Explanation: = is the assignment operator: a = 10 puts 10 into a. == is the comparison operator: a == 10 checks if a holds 10 and returns True or False. Mixing them up is one of the most common beginner bugs.

Q19. What is the output?

```
print(10 // 3 == 4)
```

A. True	**B.** False ✓
C. Error: invalid syntax	**D.** None

Explanation: 10 // 3 = 3 (floor division). Then 3 == 4 is False. Remember: // rounds DOWN to the nearest integer. 3.33 becomes 3, not 4.

Q20. In a chain of if/elif/else, how many blocks can execute?

A. All blocks where the condition is True	**B.** Only the first block — even if later ones are True ✓
C. The last matching block	**D.** All blocks always execute

Explanation: Once the FIRST matching condition executes its block, Python exits the entire if/elif/else chain. Even if 5 more conditions would be True, they are never checked. Only ONE block ever runs.

Q21. What is the output?

```
u, p = 'admin', 'xyz'
if u == 'admin' and p == '1234':
print('Correct!')
elif u != 'admin':
print('Wrong username')
elif p != '1234':
print('Wrong password')
```

A. Correct!	**B.** Wrong username ✓
C. Wrong password	**D.** Wrong secret

Explanation : With 'and', ALL conditions must be True. Username 'admin' matches, but password 'xyz' ≠ '1234', so the 'and' chain is False. Python checks elif username != 'admin' — that's False (username IS admin), so moves to next elif password != '1234' — True! So prints 'Wrong password'.

Q22. Short-circuit evaluation: if the first operand of 'and' is False, Python:

A. Still checks the second operand	**B.** Raises an error
C. Does NOT check the second operand — result is already False ✓	**D.** Returns None

Explanation : Short-circuit: with 'and', if the first condition is False, the whole expression MUST be False — no need to check the second. With 'or', if the first is True, the whole expression is True — second is skipped. This saves processing time and is an interview favourite.

Q23. What will this print?

```
a = 10
b = 10
print(a >= b)
```

A. True ✓	**B.** False
C. Error	**D.** 0

Explanation : 10 >= 10 checks two things: is 10 > 10? No (False). Is 10 == 10? Yes (True). Since ONE of them is True, >= returns True. The >= operator is internally like an OR: greater OR equal.

Q24. What does the 'not' operator do?

A. Checks if two values are not equal	**B.** Reverses the boolean value: True becomes False, False becomes True ✓
C. Stops the execution of a loop	**D.** Checks if a value is None

Explanation : not reverses the boolean. not True = False. not False = True. not (10 == 10) = not True = False. Commonly used in conditions: 'if not is_logged_in: redirect to login'.

Section 4 — Loops

Day 5 | Q25–Q31

Q25. What is the output of range(2, 10, 3)?

A. 2, 5, 8 ✓	**B.** 2, 4, 6, 8
C. 3, 6, 9	**D.** 2, 5, 8, 11

Explanation : range(start, stop, step): starts at 2, adds 3 each time, stops BEFORE 10. So: 2, 5, 8. Next would be 11 which is ≥ 10, so the loop stops. The stop value is always excluded.

Q26. What is the output?

```
for i in range(10):
print(i, end=' ')
```

A. 0 1 2 3 4 5 6 7 8 9 ✓	**B.** 1 2 3 4 5 6 7 8 9 10
C. 0 1 2 3 4	**D.** 0 2 4 6 8

Explanation : range(10) starts at 0 and goes to 9 (10 excluded). print(i, end=' ') prints with a space instead of newline. Output: 0 1 2 3 4 5 6 7 8 9.

Q27. What does 'break' do inside a loop?

A. Skips the current iteration and goes to the next one	**B.** Pauses the loop temporarily
C. Exits the loop completely, regardless of the remaining range ✓	**D.** Restarts the loop from the beginning

Explanation : break immediately exits the loop. continue skips to the next iteration. break is permanent — once triggered, the loop is done. If range was 1,000,000 and break fires at 5, it stops at 5.

Q28. What is the output?

```
for i in range(1, 11):
if i == 5:
continue
print(i, end=' ')
```

A. 1 2 3 4 6 7 8 9 10 ✓

B. 1 2 3 4 5 6 7 8 9 10

C. 1 2 3 4

D. 1 2 3 4 5

Explanation : continue at i==5 skips the print for 5 and goes to 6. The loop continues normally. So 5 is skipped but all others 1-10 print. Output: 1 2 3 4 6 7 8 9 10.

Q29. What distinguishes a while loop from a for loop?

A. while loops are faster than for loops

B. for loops can only iterate over lists; while loops work on numbers

C. while loops run based on a condition (unknown iterations); for loops iterate over a sequence (known range) ✓

D. They are identical — just different syntax

Explanation : for = use when you know HOW MANY times (range, list, etc). while = use when you repeat UNTIL A CONDITION changes. Operating systems use while loops heavily — keep running until signal received.

Q30. What happens if you forget to update the loop variable inside a while loop?

A. The loop runs exactly once and stops

B. Python automatically increments the variable

C. The loop runs forever — infinite loop ✓

D. A SyntaxError is raised immediately

Explanation : If the condition never becomes False because you never change the variable, the loop runs forever. Always ask: 'Is there something inside that will eventually make the condition False?' Use Ctrl+C to stop an infinite loop.

Q31. When does the 'else' block of a while..else execute?

A. Always, after the loop ends

B. Only when the loop exits via break

C. Only when the loop ends NATURALLY (condition became False), NOT via break ✓

D. Only when the loop runs at least once

Explanation : while..else: the else runs only if the loop completed without hitting a break. If break fires, the else is SKIPPED. Perfect for 'search until found' patterns: found → break (else skipped), not found → else prints 'Not found'.

Section 5 — Data Structures

Days 5–6 | Q32–Q38

Q32. Which data structure automatically removes duplicate values?

A. list	**B.** tuple
C. set ✓	**D.** dictionary

Explanation: A set only stores unique values. {1, 1, 2, 3} becomes {1, 2, 3}. Sets are also unordered — you cannot access by index. Use sets when you need uniqueness and don't care about order.

Q33. What is the output?

```
nums = [44, 12, 88, 5, 33, 88, 7]
nums.append(99)
print(nums)
```

A. [44, 12, 88, 5, 33, 99]	**B.** [44, 12, 88, 5, 33, 88, 7, 99] ✓
C. [99, 44, 12, 88, 5, 33, 88, 7]	**D.** Error

Explanation: append() adds to the END of the list. So 99 is added after 7. The list becomes [44, 12, 88, 5, 33, 88, 7, 99].

Q34. What is the key difference between a list and a tuple?

A. Lists allow duplicates; tuples do not	**B.** Lists are mutable (changeable); tuples are immutable (cannot be changed after creation) ✓
C. Tuples can hold more items than lists	**D.** Lists use () and tuples use []

Explanation: List = mutable ([ ]): you can add, remove, change items. Tuple = immutable (( )): once created, cannot be modified. Tuples are used for fixed data like coordinates, Django URL patterns, database records.

Q35. What is the output?

```
students = {'rahul':90,'john':95,'anita':85}
print(students.get('anita'))
```

A. None	**B.** Not found
C. KeyError	**D.** 85 ✓

Explanation: students.get('anita') returns the value for key 'anita' which is 85. .get() is safe — it returns None (or a default) if the key doesn't exist, unlike direct bracket access which raises KeyError.

Q36. What does students.get('maria', 'Not enrolled') return if 'maria' is not in the dictionary?

A. None	**B.** KeyError
C. Not enrolled ✓	**D.** False

Explanation : The second argument to .get() is the default fallback value. If the key exists, its value is returned. If not, the fallback is returned. This is production-level code pattern — always use .get() when the key might not exist.

Q37. When you iterate over a dictionary with 'for k in d:', what do you get?

A. Values only	**B.** Key-value tuples
C. Keys only ✓	**D.** Both keys and values

Explanation : By default, iterating a dictionary gives KEYS. Use d.values() for values, d.items() for (key, value) pairs. In interviews: 'for k, v in d.items()' is the Pythonic way to iterate both.

Q38. What is the output?

```
nums = [44, 12, 88, 5, 33]
nums.sort(reverse=True)
print(nums)
```

A. [5, 12, 33, 44, 88]	**B.** [88, 44, 33, 12, 5] ✓
C. [5, 12, 33, 44, 88, 7]	**D.** Error

Explanation : sort(reverse=True) sorts in DESCENDING order (largest to smallest). The original list [44, 12, 88, 5, 33] becomes [88, 44, 33, 12, 5]. sort() modifies the list in place — it doesn't return a new list.

Section 6 — Functions

Day 6 | Q39–Q40

Q39. What is the output of this code?

```
def add(a, b):
c = a + b
return c
answer = add(5, 6)
print(answer)
```

A. 11 ✓	**B.** Nothing — function is defined but never called
C. Error: function not defined	**D.** add

Explanation : The function IS called: answer = add(5, 6). 5 + 6 = 11 is stored in answer. print(answer) prints 11. Key point: the def block does nothing until called. If you removed the last two lines, there would be zero output.

Q40. Why do we use functions instead of writing the same code multiple times?

A. Functions make code run faster	**B.** Reusability — write the logic once, call it anywhere with different inputs ✓
C. Functions are mandatory in Python	**D.** To use fewer variables

Explanation : The main reason: REUSABILITY. Write once, use many times with different inputs. Also: organisation (break big programs into small named units), readability, and easier debugging. The instructor's Xerox machine analogy: same machine, different paper+content = different output.

PART 2 — One-Liner Questions & Answers

30 questions · Aim: 1 minute per question · Total: ~30 minutes

Try to answer each question before looking at the answer →

Web & Internet

Days 1–2

1. What is an API?

→ A set of rules that allows two applications to communicate with each other. Example: your app calling GitHub's server to get user data.

Note: Full form: Application Programming Interface. The word 'Interface' is key — it's the agreed contract between client and server.

2. What is the difference between an endpoint and a URL?

→ They are the same thing in most practical contexts. An endpoint is the specific address of an API service (e.g. https://api.github.com/users/john). A URL is the broader term.

Note: In interviews: endpoint emphasises it's an API address that does something specific. URL is the technical address format.

3. What does HTTPS add over HTTP?

→ HTTPS adds SSL/TLS encryption, meaning data is encrypted in transit between client and server. HTTP sends data as plain text — anyone can intercept it.

Note: S = Secure. Any real website handling passwords or payments must use HTTPS.

4. What are the two types of validation?

→ Client-side validation (in the browser, before sending to server) and Server-side validation (on the backend, after receiving the request). Both are always needed.

Note: Never rely on client-side alone — users can bypass browser checks. Server-side is the real security layer.

5. What is the purpose of requirements.txt in a Python project?

→ It lists all Python packages and their exact versions needed for the project. Anyone can run 'pip install -r requirements.txt' to install all dependencies at once with the correct versions.

Note: This ensures everyone on the team uses identical package versions — no 'works on my machine' problems.

Python Basics

Day 3

6. What is a variable in Python?

→ A named location in RAM (memory) that stores a value. Example: a = 10 creates a memory slot named 'a' holding the value 10.

Note: The name is the label, the value is what's inside. RAM stores the value during execution; disk stores the .py file.

7. How do you run a Python file from the terminal?

→ Command: python filename.py — Example: python test.py. Execution starts from line 1 (or the main function if present) and goes line by line.

Note: Python uses an interpreter — line by line. If there's an error on line 3, lines 4+ never execute.

8. What is a SyntaxError in Python?

→ An error that occurs when code breaks Python's grammar rules. Example: missing quotes, missing colon after if. Python detects this before even running the code.

Note: The error message includes the line number and a hint. Always read error messages — they tell you exactly what's wrong.

9. What is dynamic typing in Python?

→ Python automatically determines a variable's data type at runtime — you don't declare it. x = 10 is automatically an int. x = 'hello' is automatically a str.

Note: This is different from Java/C where you must write: int x = 10; Python's dynamic typing makes it flexible but requires care with type conversions.

10. What does str(10) do? And what does int('10') do?

→ str(10) converts the integer 10 to the string '10'. int('10') converts the string '10' to the integer 10. These are type conversions.

Note: int('hello') raises ValueError. int('3.5') raises ValueError. Only int('3') works for integer conversion. float('3.5') works fine.

Operators & Conditionals

Day 4

11. What is floor division? Give an example.

→ Floor division (//) divides and rounds DOWN to the nearest integer. 10 // 3 = 3 (not 3.33). 7 // 2 = 3. Even 3.99 → 3 with floor.

Note: Floor = the lower value. The // operator always gives an integer result (or float if one operand is float).

12. What is the modulus operator and when would you use it?

→ Modulus (%) returns the remainder after division. 10 % 3 = 1. Use cases: checking even/odd (n%2==0), cycling through a range, checking divisibility.

Note: n % 10 cycles 0→9→0→9. Extremely useful for circular patterns, pagination, and time calculations.

13. Explain the 'and' logical operator with an example.

→ Both conditions must be True for 'and' to return True. Example: if age >= 18 and has_id: — both must be true to proceed. If first is False, second is never checked (short-circuit).

Note: Truth table: T and T = T. T and F = F. F and T = F (short-circuits, second not checked). F and F = F.

14. What is the 'else' block in if/elif/else? Can it have a condition?

→ else is the fallback — it runs when ALL preceding if/elif conditions are False. It CANNOT have a condition. It's unconditional — if nothing else matched, else always runs.

Note: In Python, only if and elif can have conditions. else is pure fallback. You can have multiple elifs but only one else at the end.

15. What does 10 ** 3 mean in Python?

→ ** is the power/exponent operator. 10 ** 3 = 10^3 = 1000. 2 ** 8 = 256. This is equivalent to multiplying 10 by itself 3 times.

Note: In interviews: ** is Python's way of doing exponentiation. Other languages use Math.pow(10, 3) or ^ (but ^ in Python is XOR, not power!).

Loops

Day 5

16. What is the range() function? Give its three forms.

→ range(n): 0 to n-1. range(start, stop): start to stop-1. range(start, stop, step): start to stop-1 with step increment/decrement.

Note: The stop value is ALWAYS excluded. range(10) gives 0-9. To include 10: range(11). Negative step goes backwards: range(10, 0, -1).

17. What is the difference between break and continue?

→ break exits the loop completely. continue skips the rest of the current iteration and moves to the next one. break = stop. continue = skip this one, keep going.

Note: Classic use: continue to skip invalid data in a list. break to stop searching once target is found.

18. How can you iterate directly over a list in Python without using index?

→ Use 'for item in list_name:'. Example: for name in names: print(name). Python automatically moves through each element — no index needed.

Note: This is one of Python's great features. Other languages require: for(int i=0; i < arr.length; i++). Python makes it cleaner.

19. What is an infinite loop? How do you stop one?

→ A loop whose condition never becomes False — it runs forever. Example: while True: print('hello'). Stop with Ctrl+C in the terminal.

Note: Infinite loops are sometimes intentional: servers run in infinite loops listening for requests. But accidentally forgetting i+=1 in a while loop creates an unwanted infinite loop.

20. What does random.randint(1, 10) do?

→ Returns a random integer between 1 and 10, BOTH INCLUSIVE. Unlike range(), randint includes the stop value. Requires: import random.

Note: randint(a, b) → both a and b are possible results. This differs from range(a, b) which excludes b.

Data Structures

Days 5–6

21. What are Python's four built-in data structures?

→ list [ ] — ordered, mutable, allows duplicates. tuple ( ) — ordered, immutable. set { } — unordered, no duplicates. dict { } — key-value pairs.

Note: In Django development: lists for collections, tuples for fixed data (URL patterns), sets for uniqueness, dicts for API request/response data.

22. What is the difference between list.remove() and list.pop()?

→ remove(value) removes the FIRST occurrence of a specific VALUE. pop() removes and returns the LAST item. pop(index) removes by INDEX position.

Note: remove() raises ValueError if value not found. pop() raises IndexError if index out of range. pop() also RETURNS the removed item — useful to save it.

23. How do you sort a list in descending order?

→ Use nums.sort(reverse=True). This modifies the list in place. Or use sorted(nums, reverse=True) which returns a NEW sorted list.

Note: sort() changes the original list. sorted() leaves the original unchanged and returns a new list. For interviews: know both methods.

24. Why use a set instead of a list to check if an item exists?

→ Sets use hash-based lookup — checking 'x in set' is O(1) (instant, regardless of size). Lists check every element one by one — O(n) (slower as list grows).

Note: This is a performance point that impresses interviewers. For large datasets, 'x in my_set' is dramatically faster than 'x in my_list'.

25. What is the union, intersection, and difference of sets?

→ Union (A | B): all unique items from both. Intersection (A & B): only items in BOTH. Difference (A - B): items in A that are NOT in B.

Note: These are the same Venn diagram operations from school maths. Python implements them both as operators (|, &, -) and methods (.union(), .intersection(), .difference()).

26. What happens if you try to append to a tuple?

→ AttributeError: 'tuple' object has no attribute 'append'. Tuples are immutable — you cannot add, remove, or change items after creation.

Note: Immutability is intentional. In Django, tuples are used for URL patterns and CHOICES — things that should never change at runtime.

Dictionary & Functions

Day 6

27. What is the safe way to access a dictionary value and why?

→ Use dict.get('key') or dict.get('key', 'default'). This returns None (or the default) if key is missing. Direct access dict['key'] raises KeyError if the key doesn't exist.

Note: In real API backends, a missing key causes an unhandled exception and the request fails. Always use .get() for optional fields.

28. What does dict.items() return and how do you use it?

→ Returns all (key, value) pairs as tuples. Used for iterating both key and value: 'for k, v in d.items(): print(k, v)'. Most Pythonic way to loop through a dictionary.

Note: d.keys() = only keys. d.values() = only values. d.items() = both as tuples. In interviews, using .items() shows you write clean, professional code.

29. What is the difference between def and calling a function?

→ def defines the function — it is a blueprint/recipe. Calling it (function_name(args)) is when it actually EXECUTES. A function defined but never called produces ZERO output.

Note: The instructor demonstrated this live: wrote the def block, ran the file, got no output. Then added the function call — got output. def = define the machine. Call = press the start button.

30. What is the difference between a parameter and an argument?

→ Parameter: the variable name in the function DEFINITION — def add(a, b) — a and b are parameters. Argument: the actual value passed when CALLING — add(10, 20) — 10 and 20 are arguments.

Note: In casual conversation, people use these interchangeably. In interviews, knowing the precise distinction shows depth. Parameters are placeholders. Arguments are the real values.

PART 3 — Discussion Guide & Common Mistakes

~30 minutes · Review wrong answers · Discuss patterns

#	Common Mistake	Why It Happens	How to Remember
1	Using = instead of == in if conditions	Both look similar. = is assignment, == is comparison.	One = gives a value. Two == asks a question.
2	print("Hello" + 5) — TypeError	Mixing str and int with + is not allowed.	Both sides of + must be same type. Use str(5) or f-string.
3	Forgetting int(input()) — gets string not number	input() ALWAYS returns str, even for numbers.	Every number input needs int() or float() wrapping.
4	range(30, 2) expecting reverse — gets nothing	Default step is +1. 30 > 2 already, so loop never starts.	Going backwards? Always add negative step: range(30,2,-1).
5	Forgetting i += 1 in while loop — infinite loop	Nothing changes the variable, condition stays True forever.	Ask: 'What will eventually make this False?' If nothing — it's in
6	dict['key'] crashing with KeyError	Direct access assumes the key exists.	Always use dict.get('key', 'default') for optional keys.
7	Function defined but never called — nothing runs	def only defines. It must be called to execute.	def = recipe. Call = cooking. No call = no food.
8	Modifying a tuple — AttributeError	Tuples are immutable by design.	Need to change it? Use a list. Need it frozen? Use a tuple.

Quick Syntax Reference — Cheat Sheet

Topic	Syntax / Example	Notes
Variable	`x = 10  /  name = 'Rahul'`	No type declaration needed
Type check	`type(x)`	Returns <class 'int'> etc.
Type convert	`int('5')  str(10)  float('3.5')`	int(str) must be a number
Input	`x = int(input('Enter: '))`	input() always returns str
f-string	`f'Hello {name}, age {age}'`	Easiest string formatting
if/elif/else	`if cond:  elif cond2:  else:`	Only ONE block executes
and / or / not	`a>0 and b>0  /  a==1 or b==1`	and=both True, or=one True
Arithmetic	`+  -  *  /  //  %  **`	// floor, % remainder, ** power
for loop	`for i in range(n):  /  for x in lst:`	range(n)=0 to n-1
while loop	`while condition:  body  update`	Must update var or infinite!
break	`break  (inside loop)`	Exits loop immediately
continue	`continue  (inside loop)`	Skips to next iteration
list	`[1,2,3]  .append .insert .remove .pop .sort .index .count`	Mutable, ordered
tuple	`(1,2,3)  .count .index  max() min()`	Immutable — cannot change
set	`{1,2,3}  .add .remove .discard .pop  |  &  -`	No duplicates, unordered
dict	`{'k':v}  .get .keys .values .items .pop .update .clear`	Key-value pairs
function	`def name(params):  body  return val`	Nothing runs until called
API call	`import requests  /  requests.get(url).json()`	JSON → dict via .json()